

Java程序设计基础

Java Programming Fundamentals

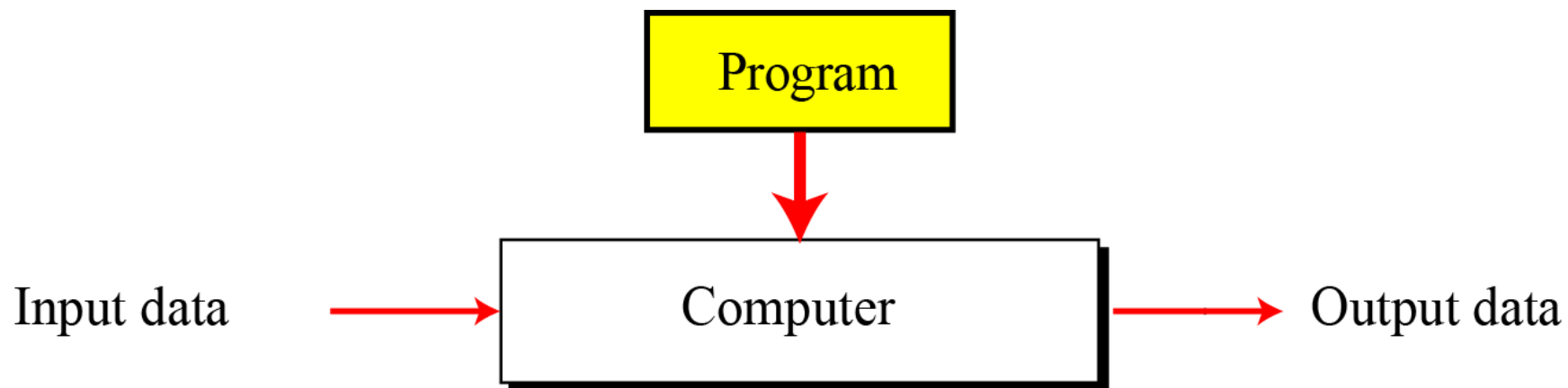
程序与内存模型



目录

- 冯·诺依曼模型
- 程序-指令-语言
- 程序执行过程分析
- 程序的内存模型

图灵机：数据处理模型

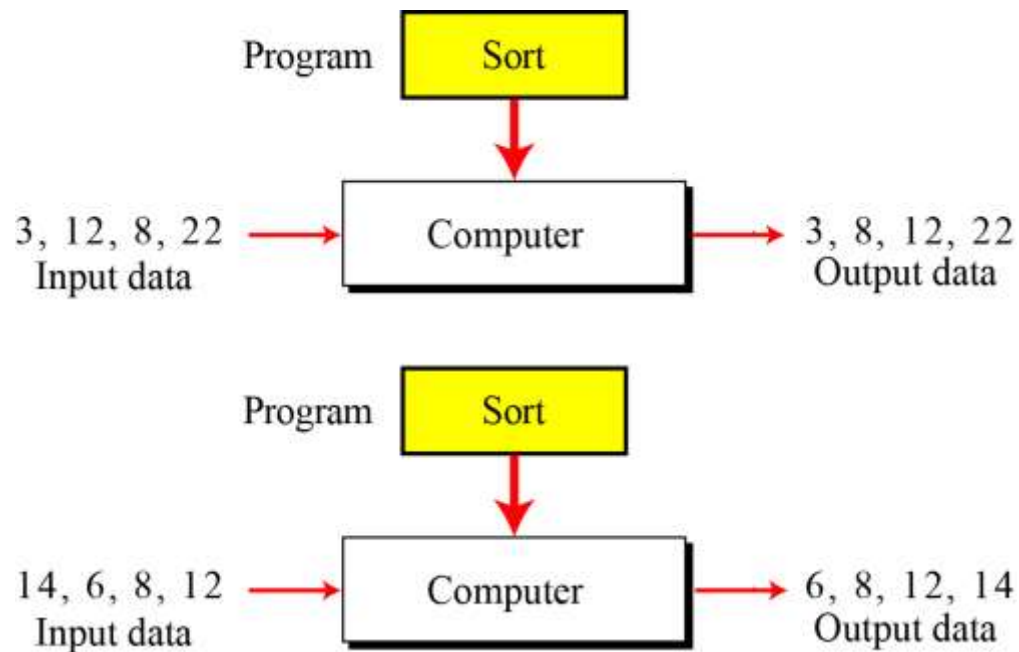


A computer based on the Turing model

程序与数据的关系

- 程序的通用性

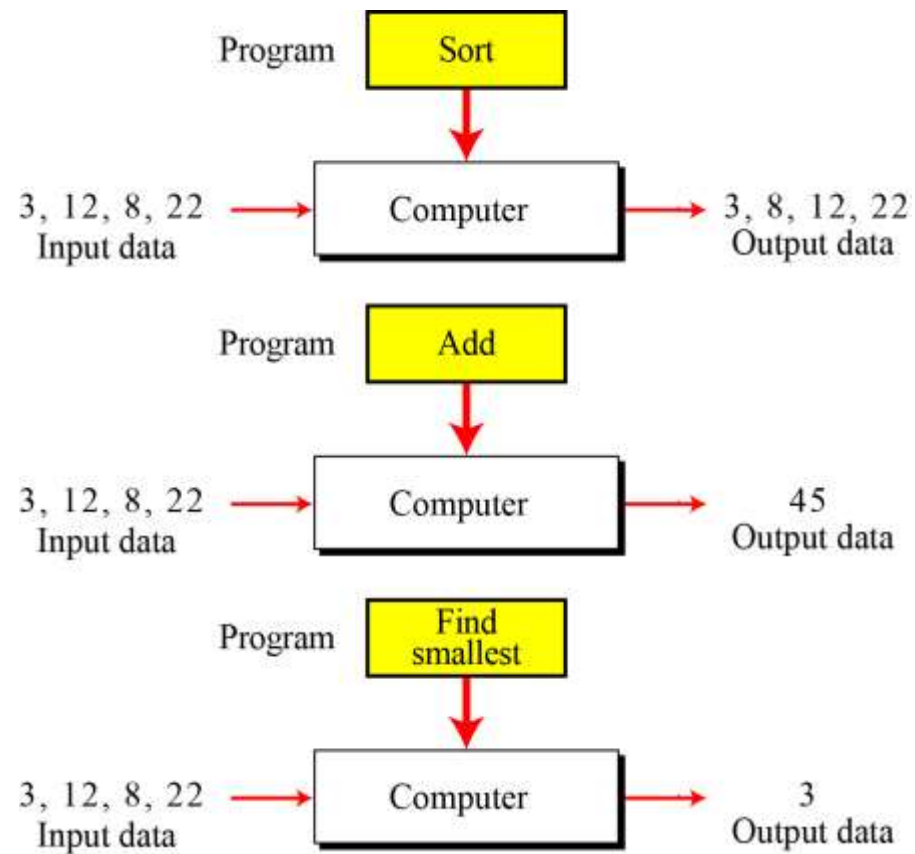
- 相同的程序可以处理不同的数据



程序与数据的关系

- 相同的数据

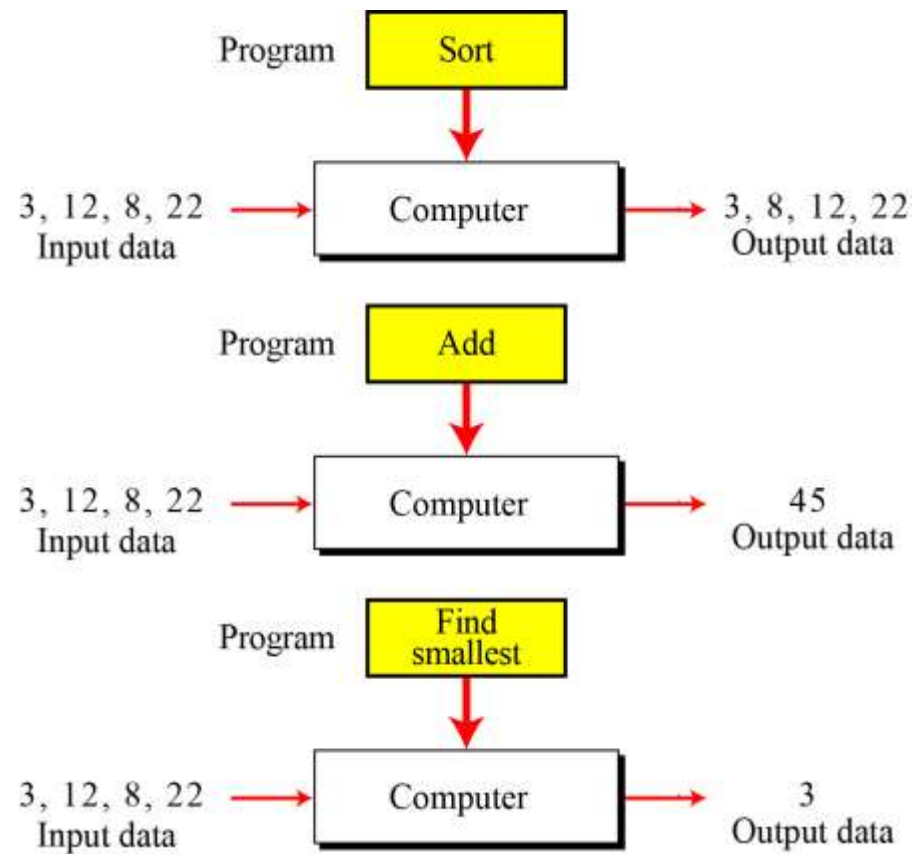
- 不同的程序
- 不同的结果



程序与数据的关系

- 相同的数据

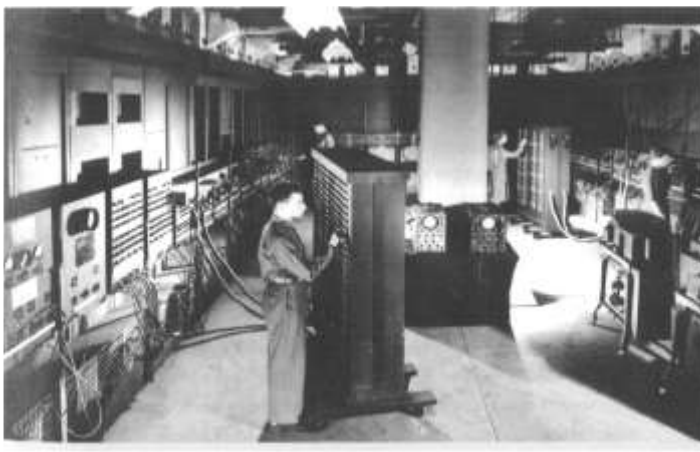
- 不同的程序
- 不同的结果



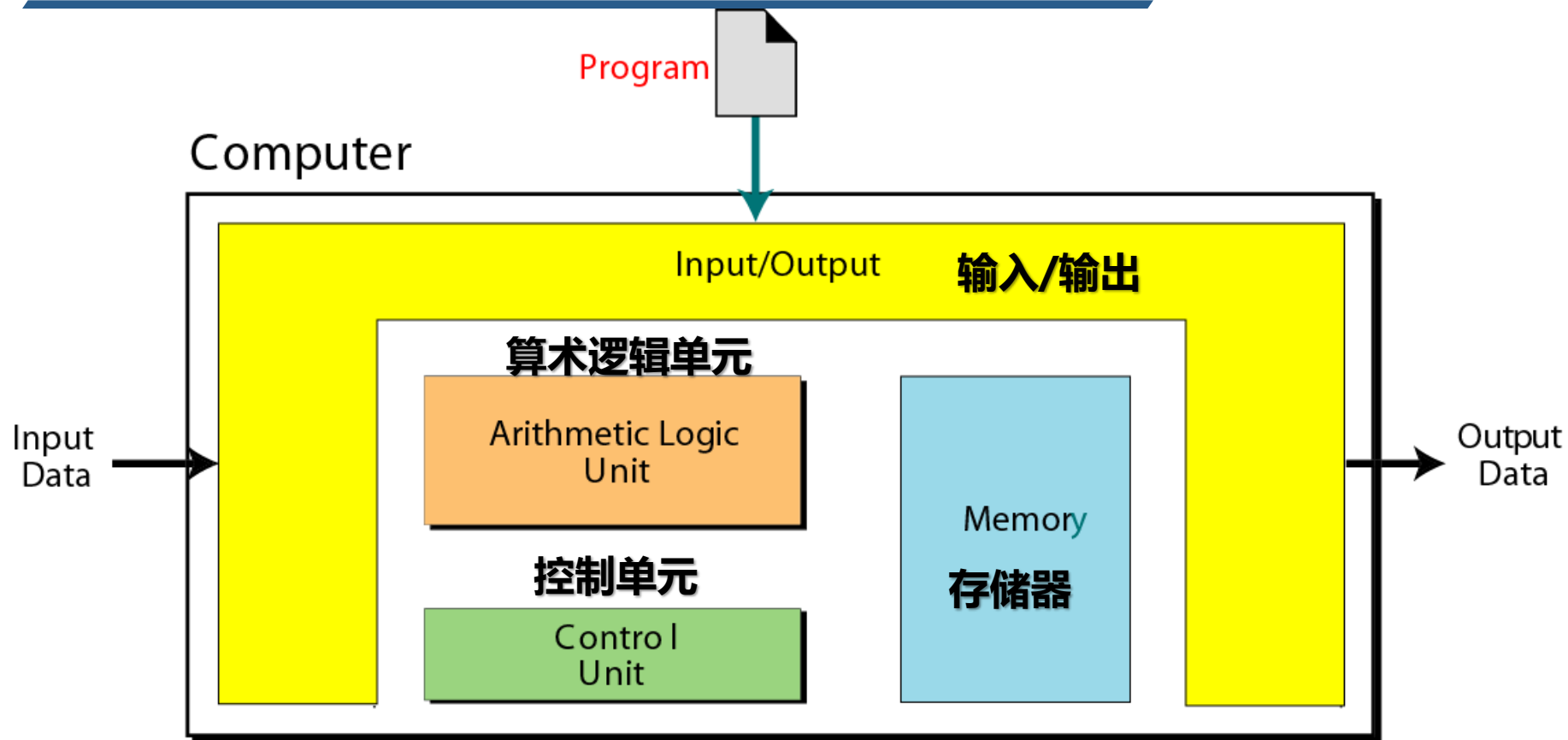
ENIAC: 大家公认的第一台电子计算机

1946年2月14日，**世界上第一台电子计算机“ENIAC”**终于研制成功。它采用穿孔卡输入输出数据，总共安装了17468只电子管，7200个二极管，70000多电阻器，10000多只电容器和6000只继电器；占地面积为170平方米左右，总重量达到30吨。ENIAC的运算速度达到每秒钟5000次加法。一条炮弹的轨迹，20秒钟就能被它算完，比炮弹本身的飞行速度还要快。

ENIAC标志着电子计算机的创世，
人类社会从此大步迈进了电脑时代的门槛。

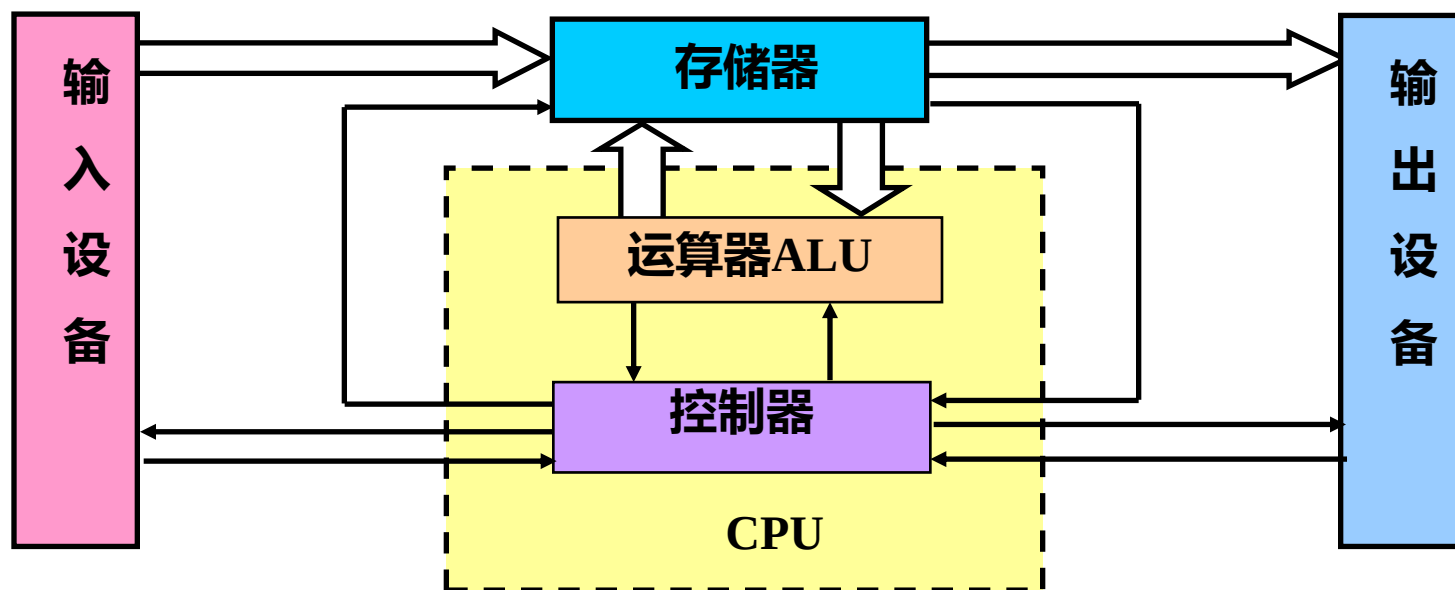


冯.诺伊曼模型(von Neumann model)



二进制：简化存储器和运算器结构
存储程序：把程序和数据都存储到存储器中
程序控制：自动执行、避免干扰、保证速度

冯.诺伊曼模型(von Neumann model)



“存储程序” + “程序控制”
计算机的两个基本能力：

能够存储程序

能够自动地执行程序

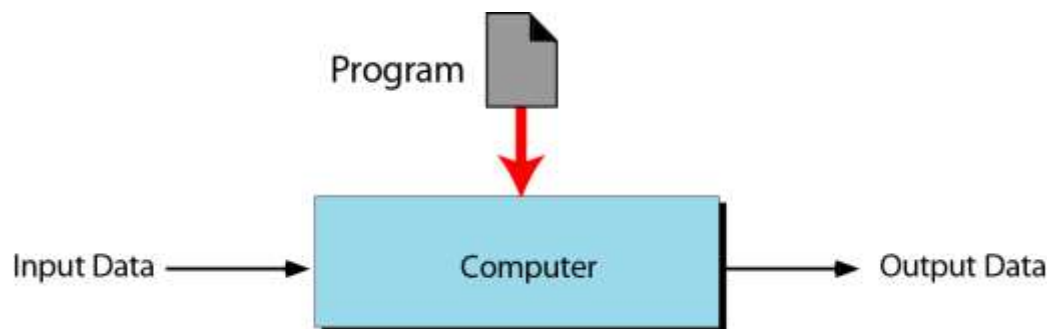
将程序和数据统一采用
二进制数据表示

五大部件

目录

- 冯·诺依曼模型
- 程序-指令-语言
- 程序执行过程分析
- 程序的内存模型

> 计算机与程序



1. Input first data item into memory.
2. Input second data item into memory.
3. Add the two together and store the result in memory.
4. Output the result.

Program

程序-指令-语言

Table 9.1 Code in machine language to add two integers

Hexadecimal	Code in machine language			
(1FEF) ₁₆	0001	1111	1110	1111
(240F) ₁₆	0010	0100	0000	1111
(1FEF) ₁₆	0001	1111	1110	1111
(241F) ₁₆	0010	0100	0001	1111
(1040) ₁₆	0001	0000	0100	0000
(1141) ₁₆	0001	0001	0100	0001
(3201) ₁₆	0011	0010	0000	0001
(2422) ₁₆	0010	0100	0010	0010
(1F42) ₁₆	0001	1111	0100	0010
(2FFF) ₁₆	0010	1111	1111	1111
(0000) ₁₆	0000	0000	0000	0000

Table 9.2 Code in assembly language to add two integers

Code in assembly language	Description
LOAD RF Keyboard	Load from keyboard controller to register F
STORE Number1 RF	Store register F into Number1
LOAD RF Keyboard	Load from keyboard controller to register F
STORE Number2 RF	Store register F into Number2
LOAD R0 Number1	Load Number1 into register 0
LOAD R1 Number2	Load Number2 into register 1
ADDI R2 R0 R1	Add registers 0 and 1 with result in register 2
STORE Result R2	Store register 2 into Result
LOAD RF Result	Load Result into register F
STORE Monitor RF	Store register F into monitor controller
HALT	Stop

Program 9.1 Addition program in C++

```

/* This program reads two integers from keyboard and prints their sum.
   Written by:
   Date:

*/
#include <iostream.h>
using namespace std;
int main (void)
{
    // Local Declarations
    int number1;
    int number2;
    int result;
    // Statements
    cin >> number1;
    cin >> number2;
    result = number1 + number2;
    cout << result;
    return 0;
} // main

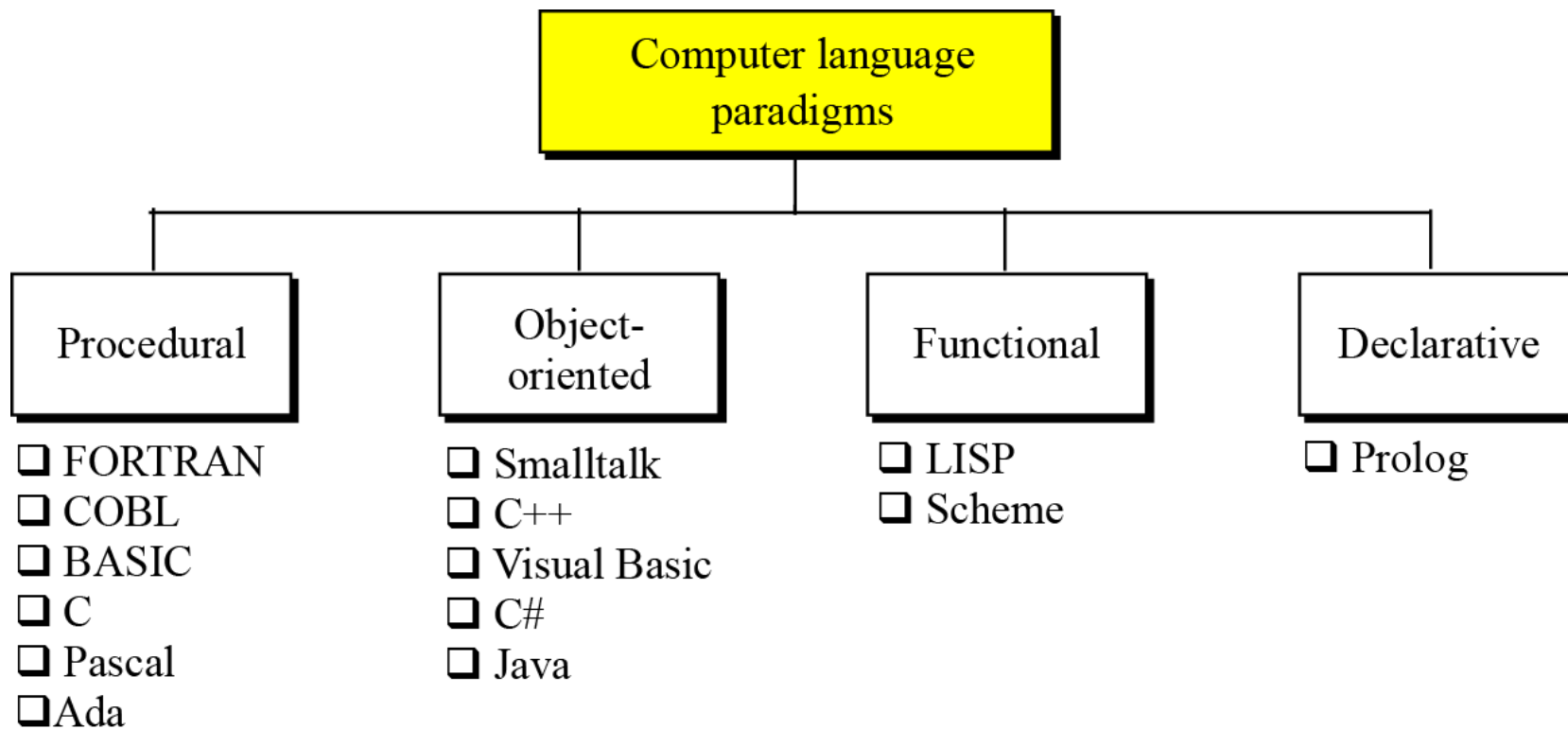
```

指令集(Instruction Set)

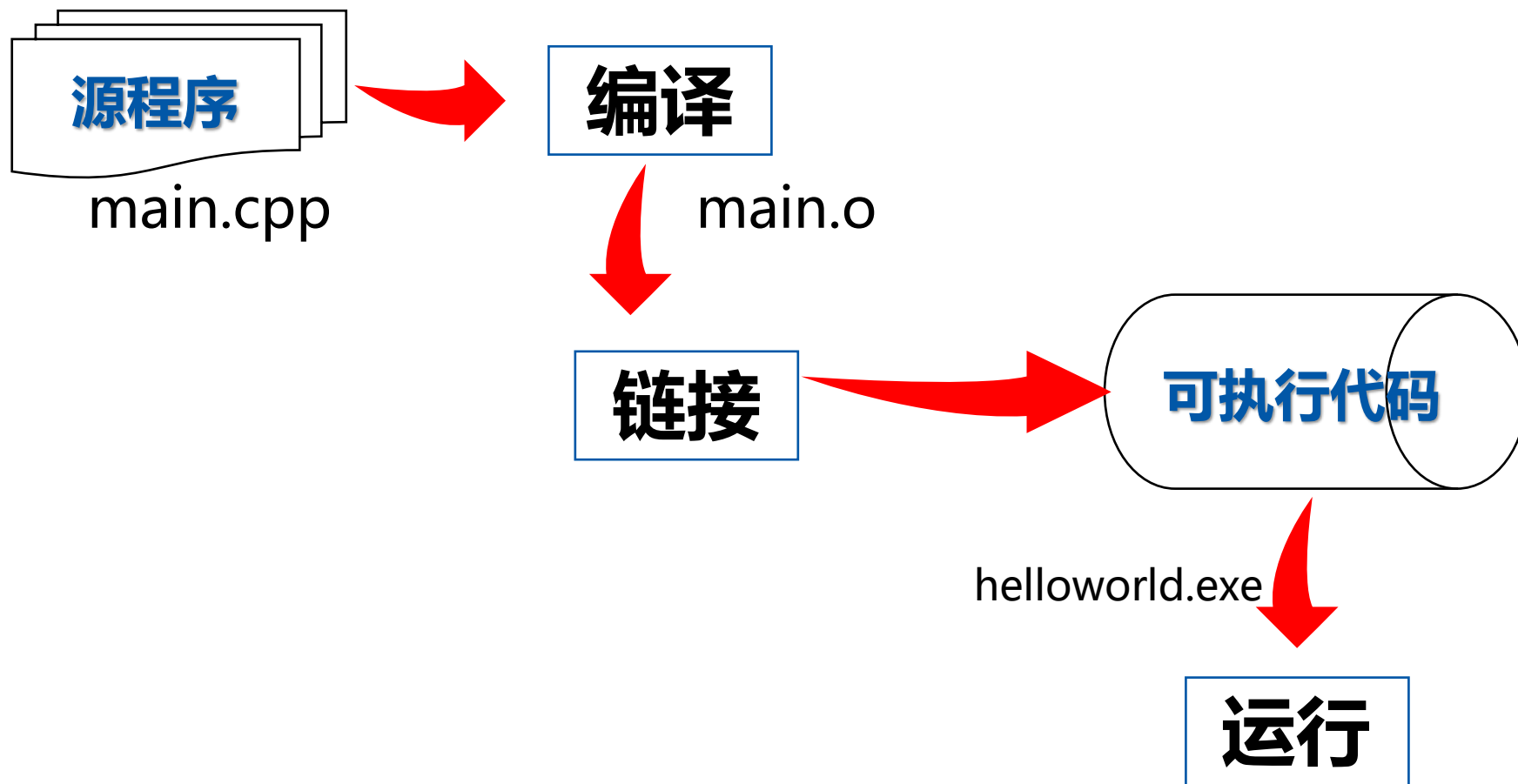
- **CPU所支持的指令的集合**
 - 所有的语言最终都要翻译为指令集中的指令才能执行
 - 定义了计算机所能支持的所有操作
- **有哪些指令集**
 - LoongArch指令集
 - X86指令集
 - MIPS指令集
 - ARM指令集
 -



语言的分类



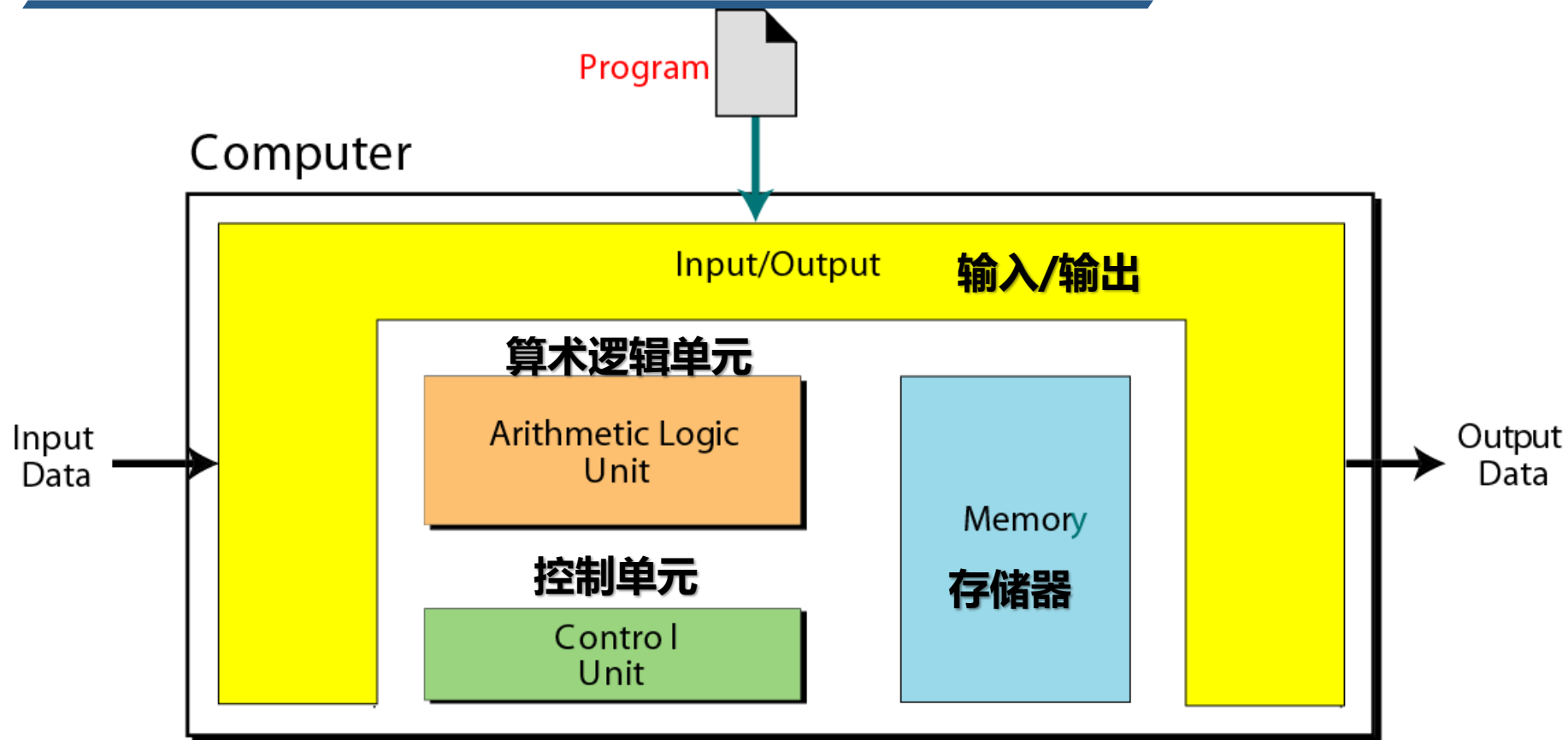
源代码→可执行程序



目录

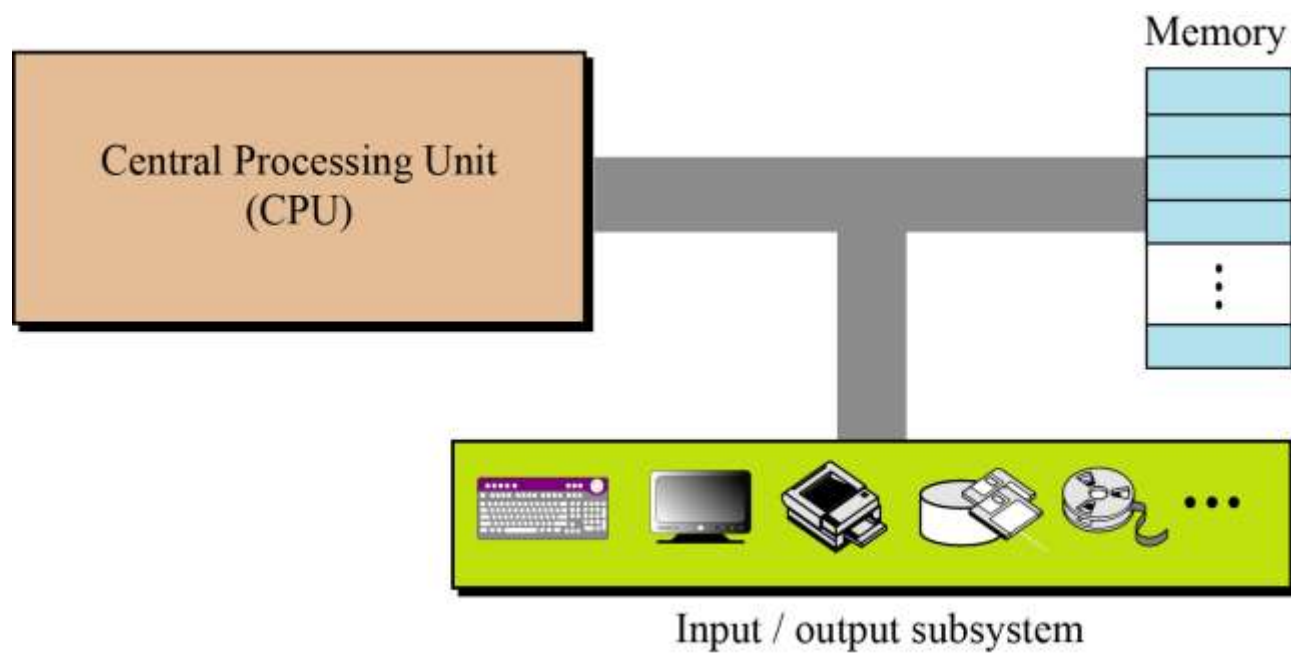
- 冯·诺依曼模型
- 程序-指令-语言
- 程序执行过程分析
- 程序的内存模型

冯.诺伊曼模型(von Neumann model)

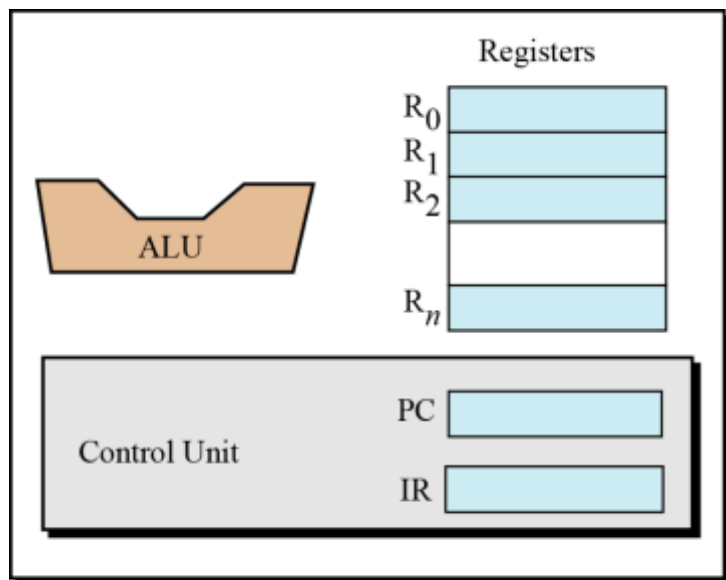


二进制：简化存储器和运算器结构
存储程序：把程序和数据都存储到存储器中
程序控制：自动执行、避免干扰、保证速度

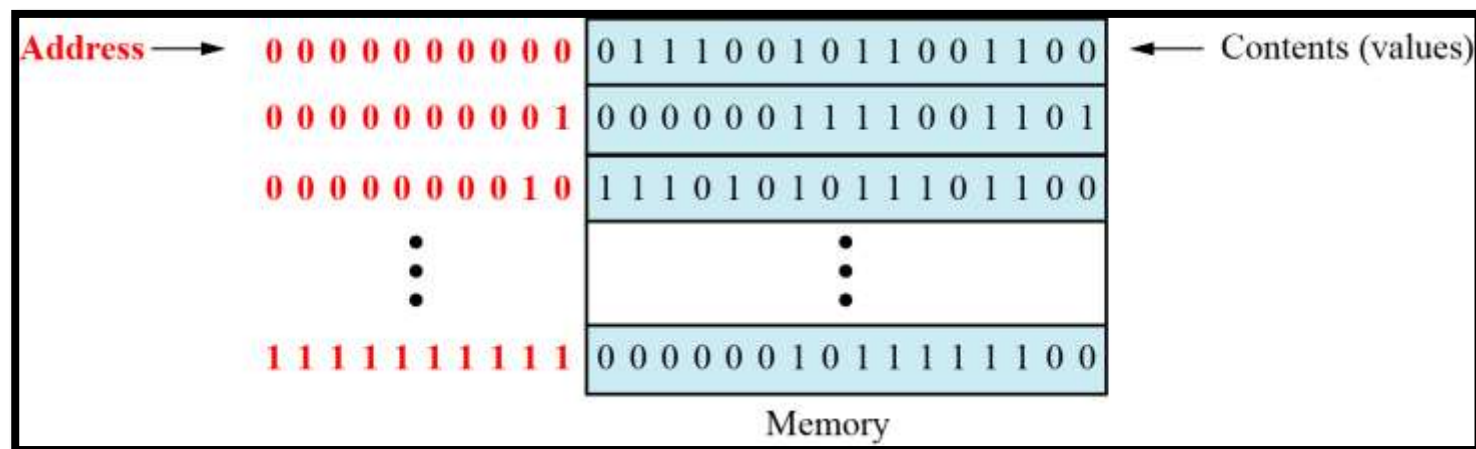
计算机硬件子系统



CPU与主存

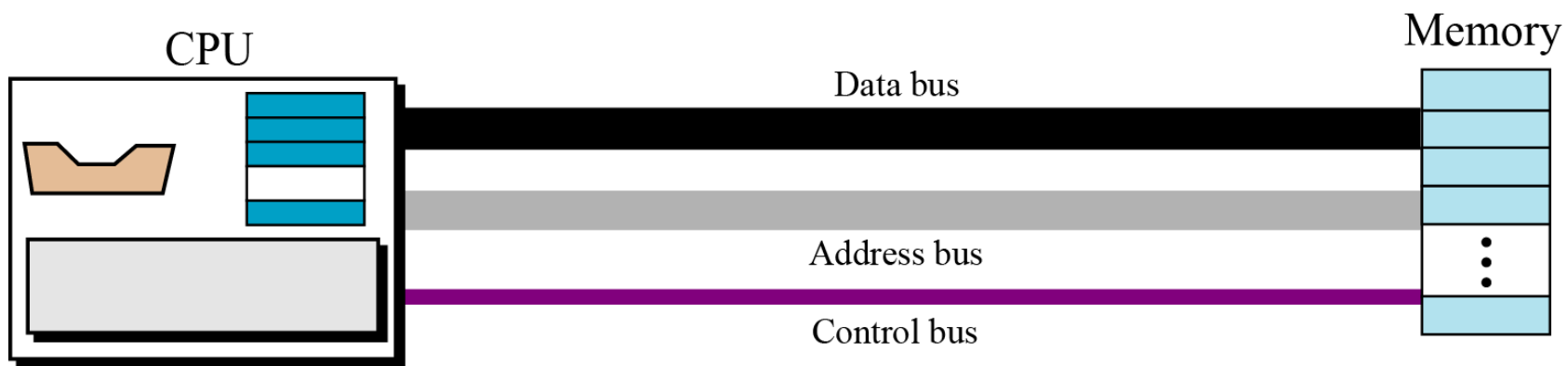


Central Processing Unit (CPU)

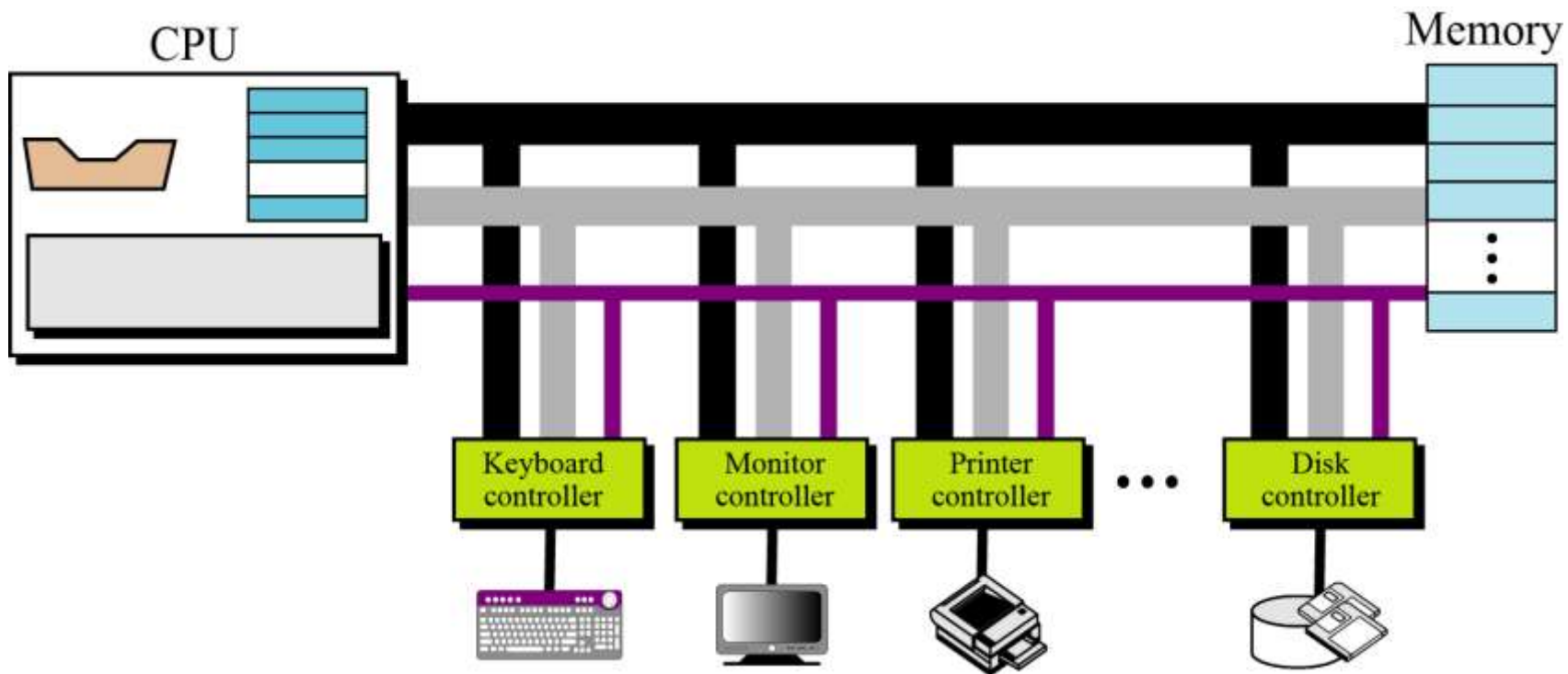




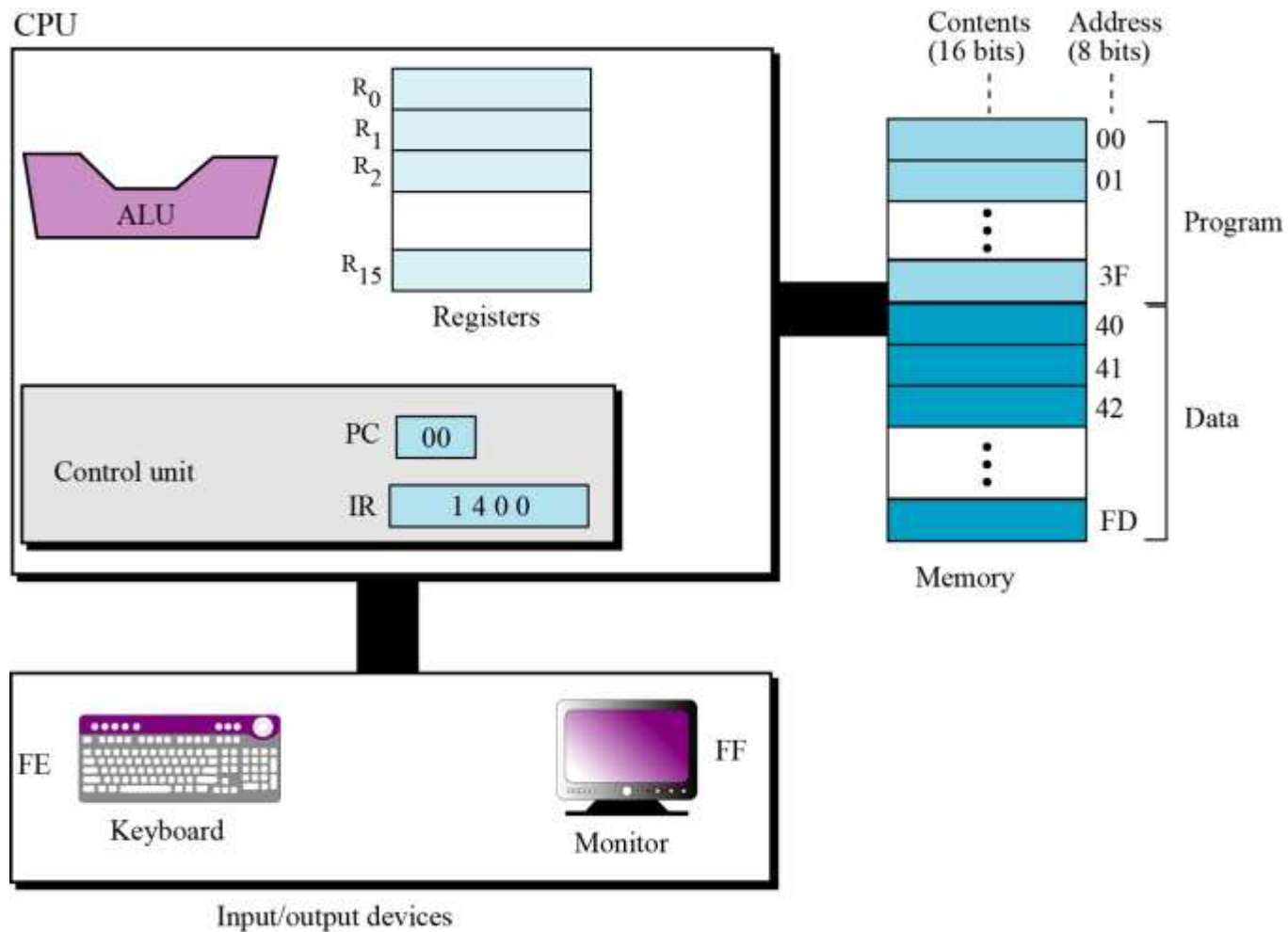
CPU与主存的连接



CPU、主存与外部设备的连接



模型机简介





模型机指令集

- 支持15条指令

- HALT - 停机
- LOAD - 装载
- STORE - 存回
- ADDI - 加法
- ADDF - 浮点数加法
- . . .



Table 5.4 List of instructions for the simple computer

Instruction	Code	Operands			Action
	d ₁	d ₂	d ₃	d ₄	
HALT	0				Stops the execution of the program
LOAD	1	R _D	M _S		R _D ← M _S
STORE	2	M _D		R _S	M _D ← R _S
ADDI	3	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
ADDF	4	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
MOVE	5	R _D	R _S		R _D ← R _S
NOT	6	R _D	R _S		R _D ← $\overline{R_S}$
AND	7	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} AND R _{S2}
OR	8	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} OR R _{S2}
XOR	9	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} XOR R _{S2}
INC	A	R			R ← R + 1
DEC	B	R			R ← R – 1
ROTATE	C	R	n	0 or 1	Rot _n R
JUMP	D	R	n		IF R ₀ ≠ R then PC = n, otherwise continue
Key: R _S , R _{S1} , R _{S2} : Hexadecimal address of source registers R _D : Hexadecimal address of destination register M _S : Hexadecimal address of source memory location M _D : Hexadecimal address of destination memory location n: hexadecimal number d ₁ , d ₂ , d ₃ , d ₄ : First, second, third, and fourth hexadecimal digits					



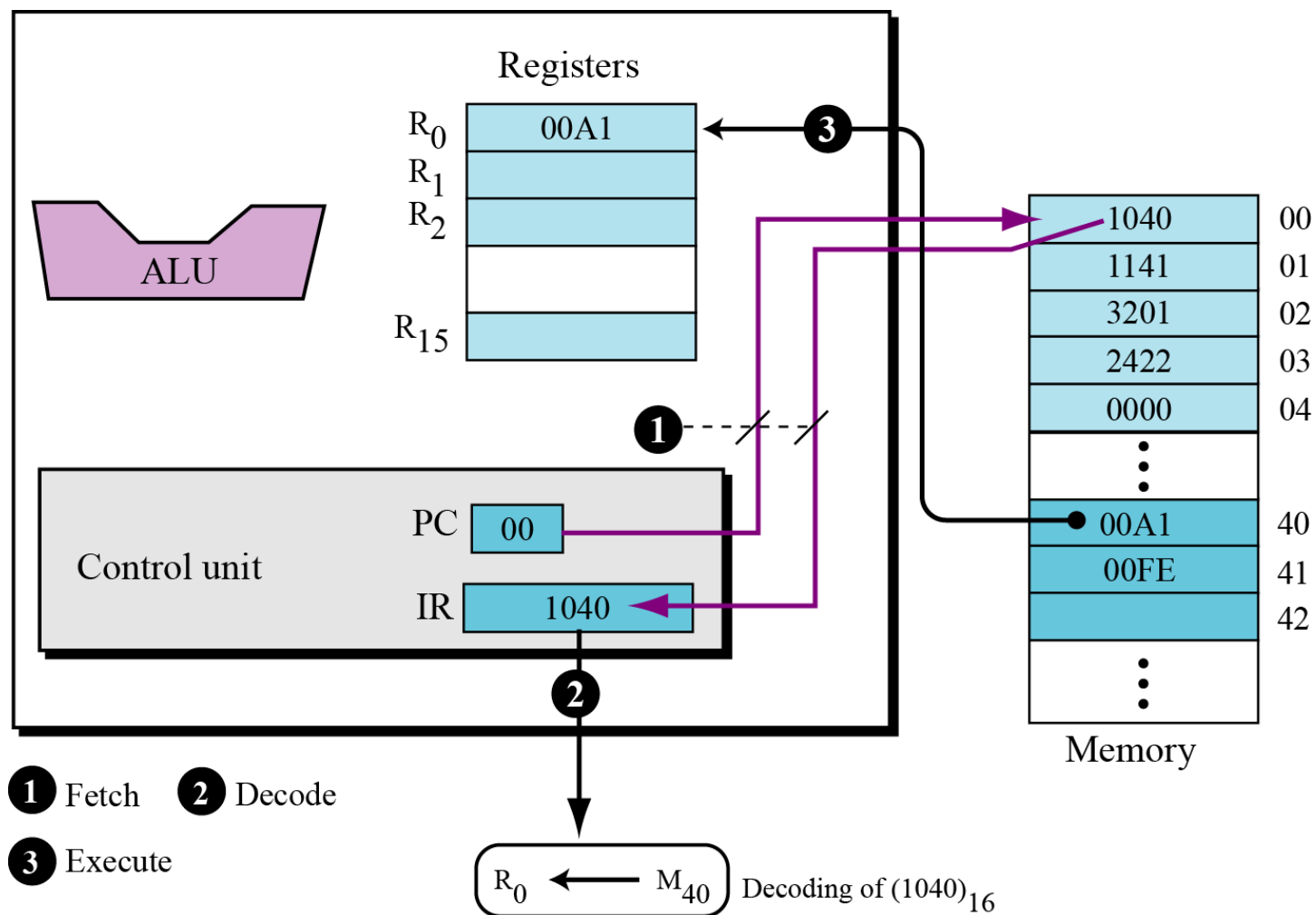
计算任务:

$$C = A + B$$

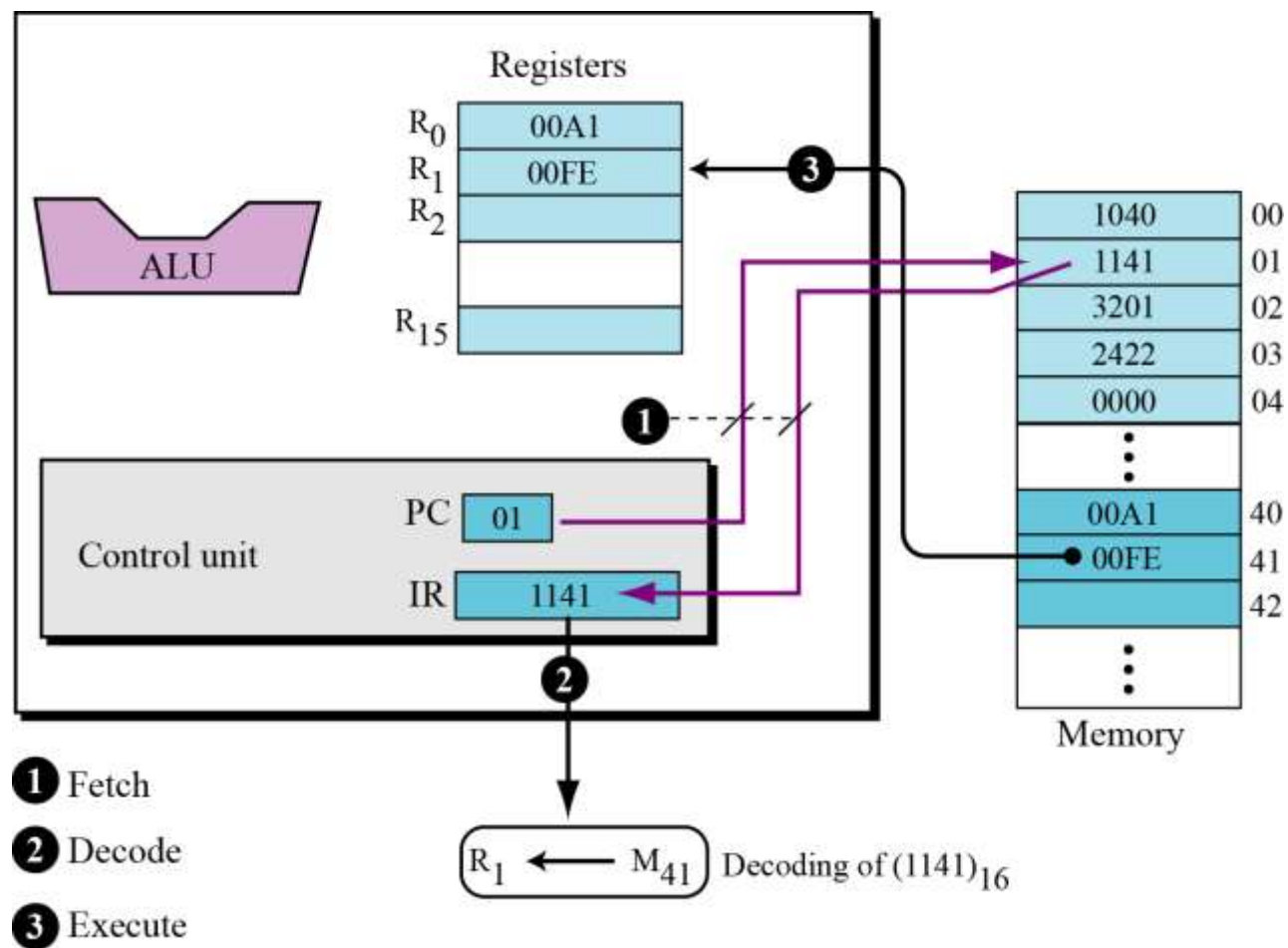
1. Load the contents of M_{40} into register R_0 ($R_0 \leftarrow M_{40}$).
2. Load the contents of M_{41} into register R_1 ($R_1 \leftarrow M_{41}$).
3. Add the contents of R_0 and R_1 and place the result in R_2 ($R_2 \leftarrow R_0 + R_1$).
4. Store the contents R_2 in M_{42} ($M_{42} \leftarrow R_2$).
5. Halt.

Code	Interpretation			
$(1040)_{16}$	1: LOAD	0: R_0	40: M_{40}	
$(1141)_{16}$	1: LOAD	1: R_1	41: M_{41}	
$(3201)_{16}$	3: ADDI	2: R_2	0: R_0	1: R_1
$(2422)_{16}$	2: STORE	42: M_{42}		2: R_2
$(0000)_{16}$	0: HALT			

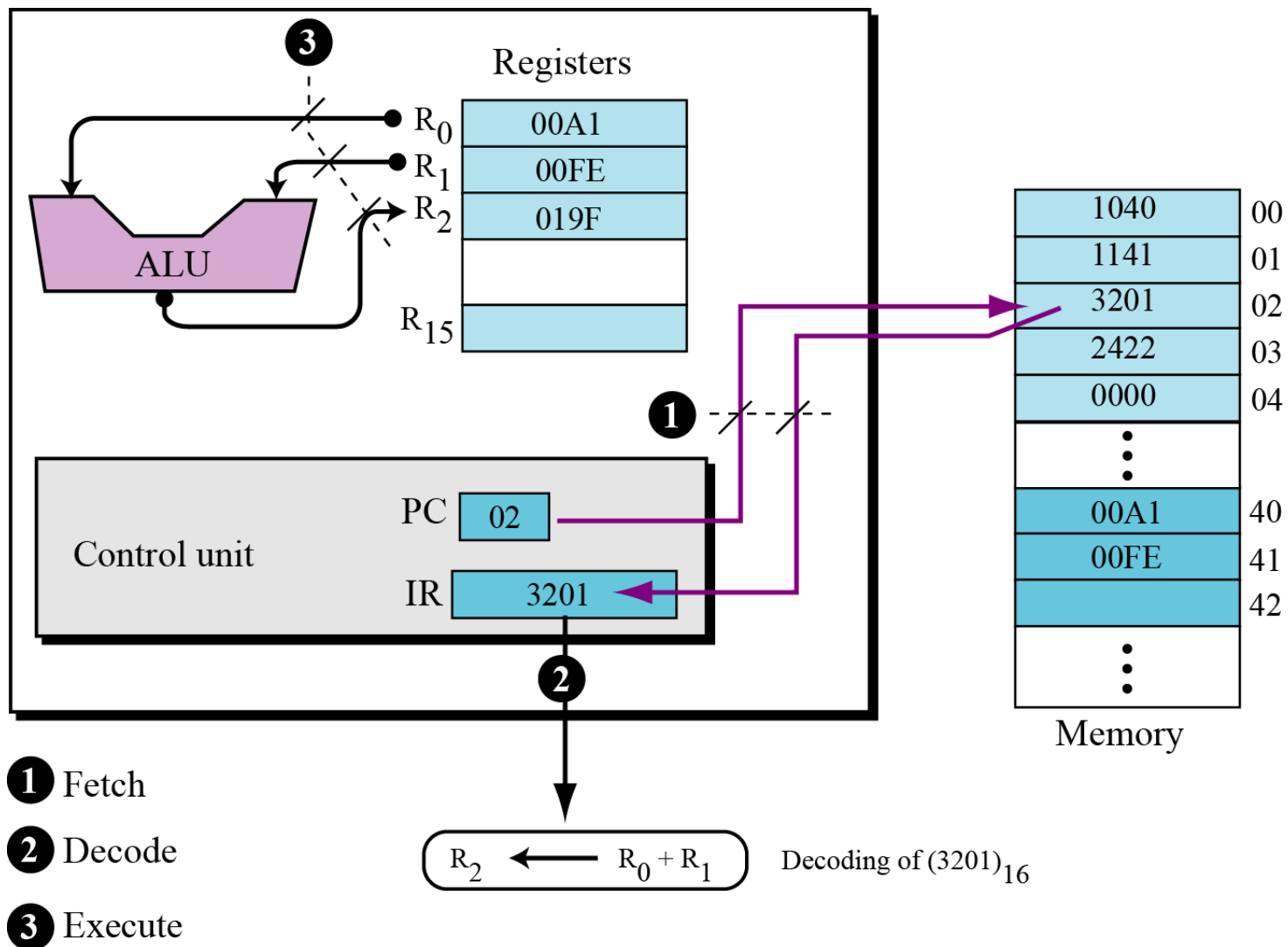
Step 1



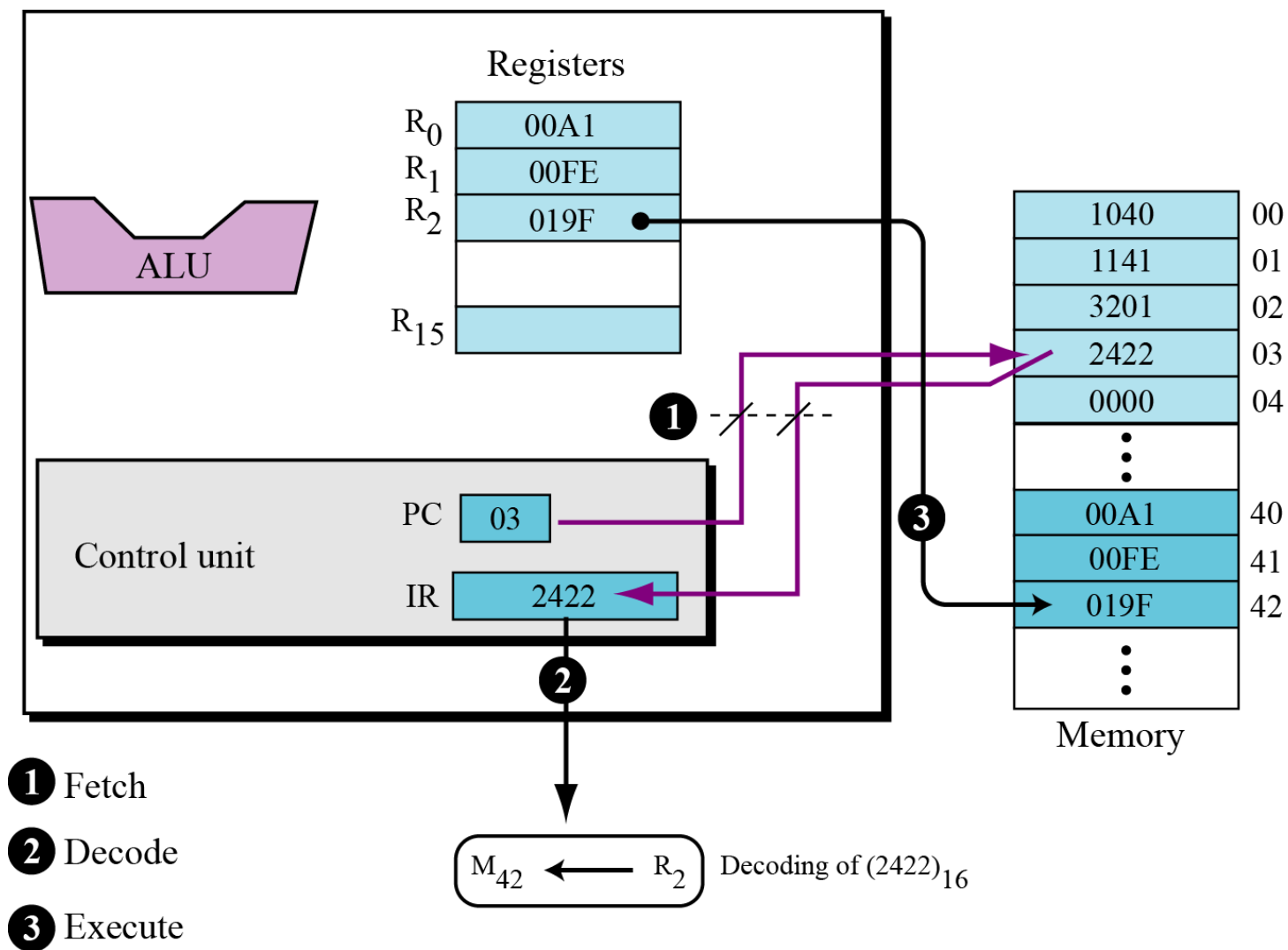
Step 2



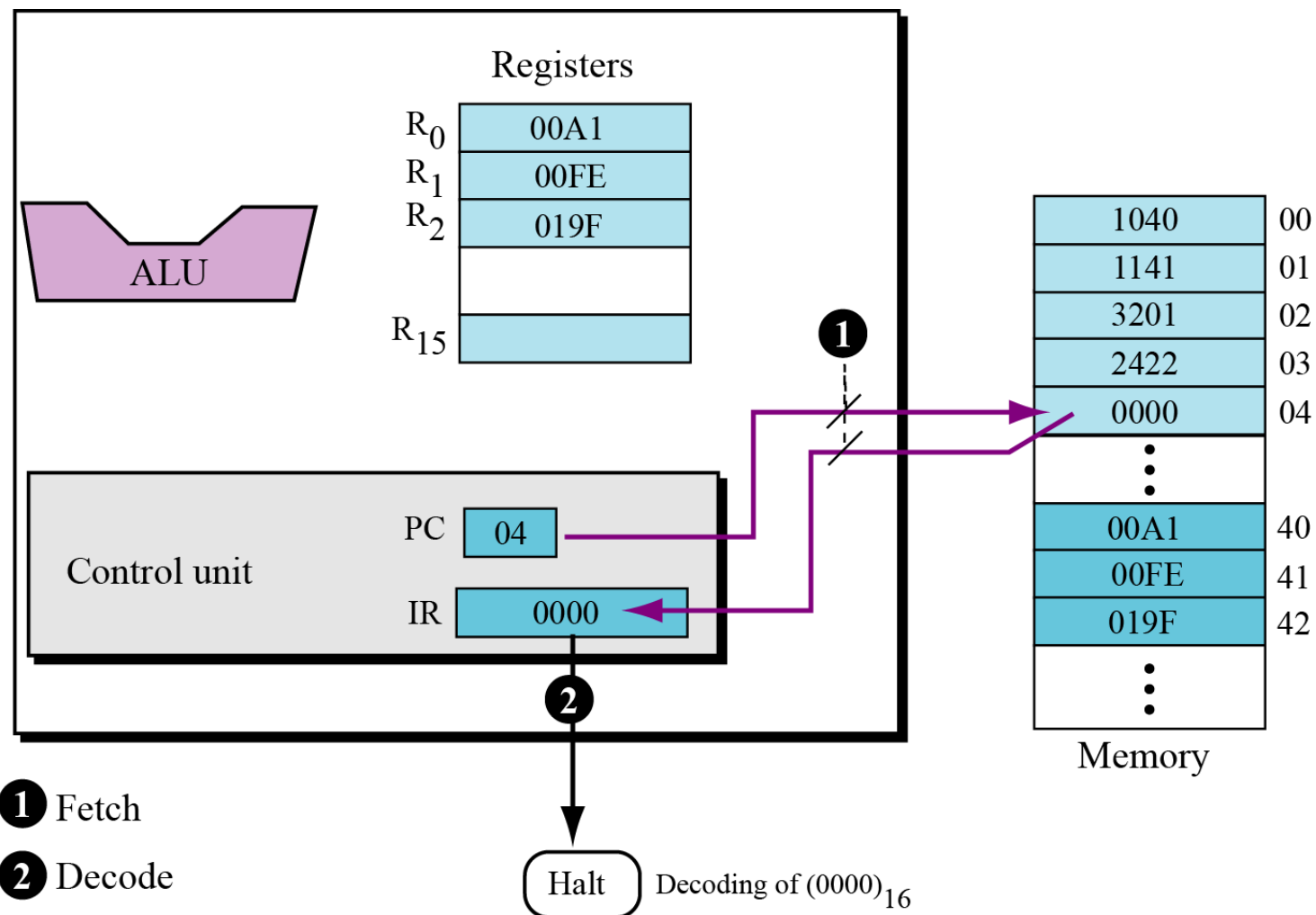
Step 3



Step 4



Step 5



目录

- 冯·诺依曼模型
- 程序-指令-语言
- 程序执行过程分析
- 程序的内存模型

程序的本质

- 程序就是通过一系列指令来对内存中的数据进行处理。
- 数据在哪里？
 - 内存中
- 如何访问（读/写）数据？
 - 通过数据的存放地址（内存地址）
- 编写程序的任务
 - 做什么操作？ -- 指令集中的指令
 - 要操作的数据在那个位置？ -- 数据在内存的哪个地址
 - 如何表示这个位置？ -- 变量

变量的本质

- **变量是一段连续内存空间的别名**
 - 程序通过变量来申请和命名内存空间
 - 程序通过变量名访问内存空间
 - 不是向变量名读写数据，而是向变量所代表的内存空间中读写数据。
- 变量代表的内存空间到底是多大？ --- **数据类型**

char : ASCII

整型 : 原码和补码

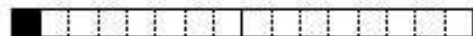
浮点型 : IEEE754方案 (包含移码)



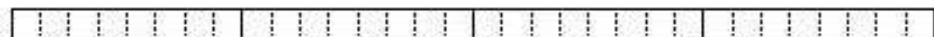
char



short



unsigned int

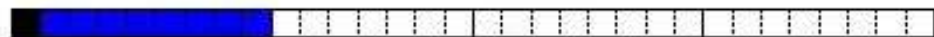


signed int



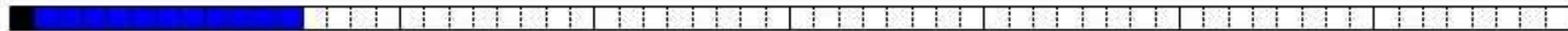
// 首位是符号位

float



// 首位是符号位, 然后是指数位8位, 剩下的是小数位

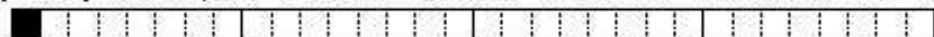
double



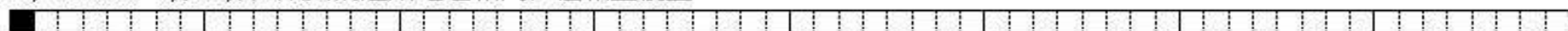
short arr[3]



enum Nm{LOSS, DRAW, WIN}nm; // 实质是一个整型, 成员只是可能的右值 (符号常量) 的枚举



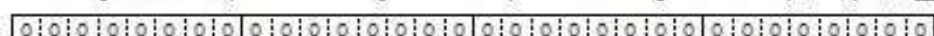
union Nm{int a; double b;}nm; // 成员的起始地址相同, 值相互覆盖



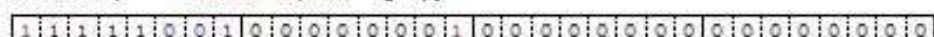
struct demo{char a; short b; int c;} abc; // 成员相对于基址偏移, 字节对齐



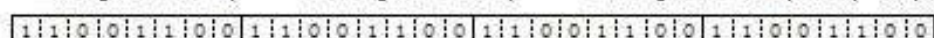
struct Bf{ unsigned a:3; unsigned b:4; unsigned c:5;}bf; //全局, 会初始化为0



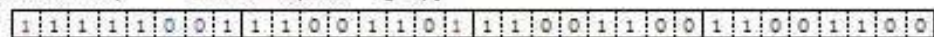
bf.a =1; bf.b=15; bf.c=3; // 小端



struct Bf{ unsigned a:3; unsigned b:4; unsigned c:5;}bf; //局部, 局部变量在Debug模式下每个字节会初始化为CC



bf.a =1; bf.b=15; bf.c=3; // 小端



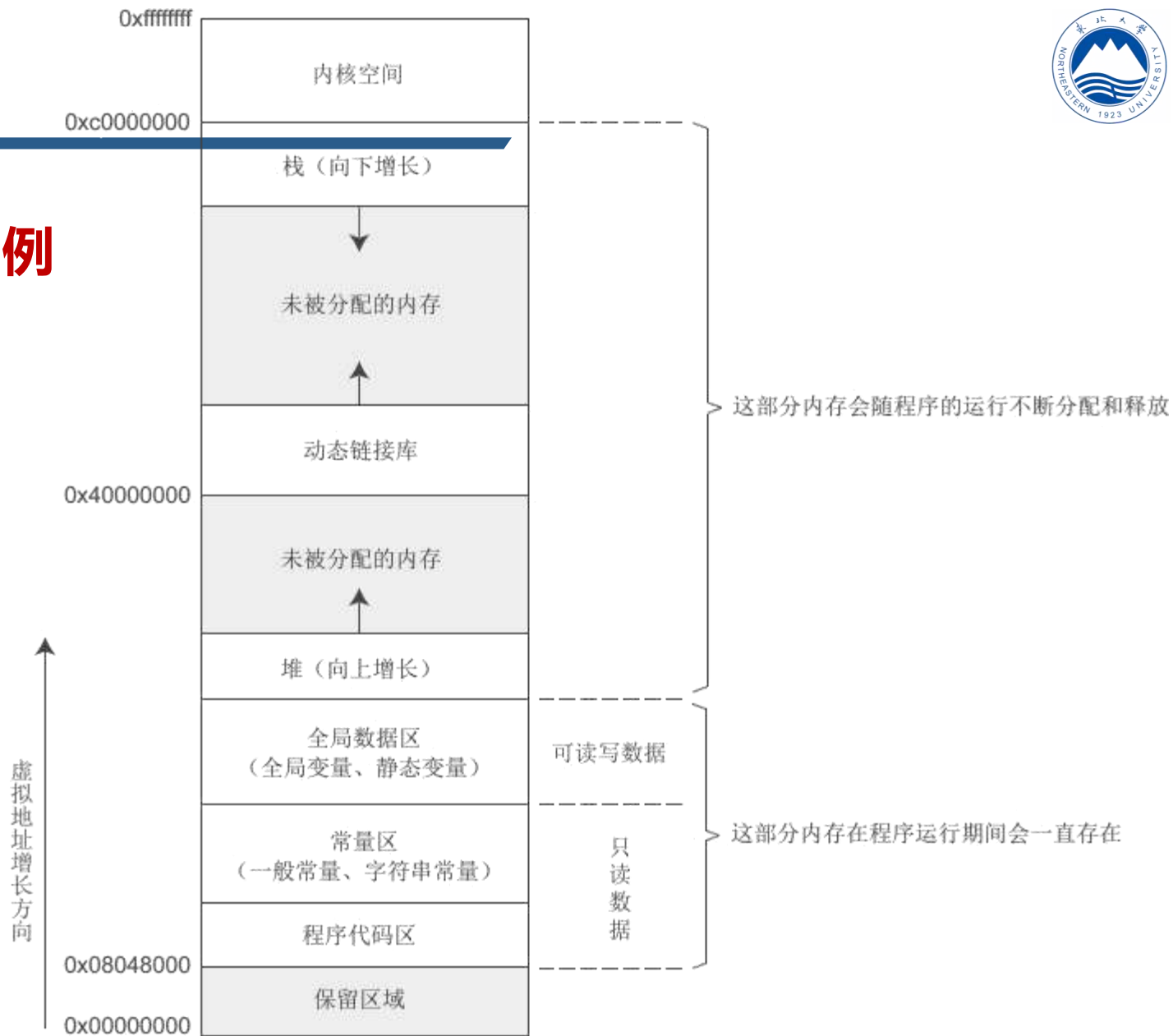
数据类型的本质

- 数据类型是固定大小内存的别名
- 为什么要分不同的数据类型?
 - 满足编程时数据对空间的需求
- 变量地址 + 变量类型
 - 可以精确的取出固定长度的内存
 - 到底是什么? -- 解释方法
- 举例: int -- float 都为4个字节
 - float的4字节解释: 符号 + 指数 + 尾数
- 类比: 给动物造窝
 - 造小了一住不下
 - 造大了浪费空间
 - 狗窝 - 大象窝
- 类比: SKU单位 (StockKeepingUnit)
 - 吨 - 公斤 - 克 - 毫克

内存模型

• 以Linux32系统内存模型为例

- 保留区
- 程序代码区
- 常量区
- 全局数据区
- 堆区
- 动态链接库
- 栈区
- 内核空间



Linux32系统内存模型

内存分区	说明
程序代码区 (code)	存放函数体的二进制代码。一个C语言程序由多个函数构成，C语言程序的执行就是函数之间的相互调用。
常量区 (constant)	存放一般的常量、字符串常量等。这块内存只有读取权限，没有写入权限，因此它们的值在程序运行期间不能改变。
全局数据区 (global data)	存放全局变量、静态变量等。这块内存有读写权限，因此它们的值在程序运行期间可以任意改变。
堆区 (heap)	一般由程序员分配和释放，若程序员不释放，程序运行结束时由操作系统回收。malloc()、calloc()、free() 等函数操作的就是这块内存，这也是本章要讲解的重点。 注意：这里所说的堆区与数据结构中的堆不是一个概念，堆区的分配方式倒是类似于链表。
动态链接库	用于在程序运行期间加载和卸载动态链接库。
栈区 (stack)	存放函数的参数值、局部变量的值等，其操作方式类似于数据结构中的栈。

程序的加载与执行过程

• 程序编写与编译

- 编辑→编译→链接 →→ 可执行程序文件

• 程序执行 -- 操作系统的职责

- 创建进程→分配空间→完成加载（磁盘文件→内存）→就绪态
- 等待操作系统调度，从main()开始执行

```
1  #include <iostream>
2  using namespace std;
3
4  int sum(int x,int y)
5  {
6      return x+y;
7  }
8
9  int sum_ab(int x,int y)
10 {
11     if (x>y) swap(x,y);
12     int sum=0;
13     for(int i=x; i<=y; i++)
14         sum = sum + i;
15
16     return sum;
17 }
18
19 int main()
20 {
21     int n,a,b;
22     cin>>n;
23
24     for(int i=1; i<=n; i++)
25     {
26         cin>>a>>b;
27         cout<<sum(a,b)<<" "<<sum_ab(a,b)<<endl;
28     }
29     return 0;
30 }
31
32
```

1 包含头文件iostream
声明名字空间std

2 定义函数sum，求边界值的和

3 定义函数sum_ab，求区间累加和

4 主函数main

5 调用函数sum

6 调用函数sum_ab

本章小结

- 冯·诺依曼模型
- 程序-指令-语言
- 程序执行过程分析
- 程序的内存模型